

1. Project overview

In this project, you will simulate a *solar tracking system*, a mechanical solar panel controlled by a control system to automatically rotate and follow the sun's path across the sky. This is indeed simulating how a real-world renewable energy system maximizes its power capture. First, you will model the mechanical system using Gazebo (Ignition) and, then, you will design a control system so that the solar panel can effectively follow the sun's path. You will then evaluate the performance of the control system based on its tracking error.

2. Gazebo Model

First, you need to build the mechanical system using Gazebo. In this regard, you will build the physical simulation of the solar tracking system using Simulation Description Format (SDF) in Gazebo. In order to do this, you need to set up the simulation environment, the mechanical links, the joints and kinematics, and the actuation and sensing plugins.

2.1. The Simulation Environment

The simulation environment defines the physical world (named *tracker_world*) where the solar tracker operates and includes basic physics settings, lighting, and the ground surface. These elements ensure the virtual world behaves predictably and provides a visual context for the tracker's movements. For this purpose, you should consider the following conditions in terms of lighting and ground.

- Physics: The simulation uses a physics time step of *0.001* seconds with a real-time factor of *1.0* to ensure the simulation runs in real time with high temporal resolution.
- World Plugins: Three essential world-level plugins are loaded to support the simulation, which are the Physics plugin (*gz-sim-physics-system*) to handle rigid-body dynamics, the Scene Broadcaster plugin (*gz-sim-scene-broadcaster-system*) to stream visual scene data, and the User Commands plugin (*gz-sim-user-commands-system*) to allow runtime interactions with the simulation.
- Lighting: The world includes a directional light source named *sun* positioned at *(0, 0, 10)* with a direction of *(-0.5, 0.5, -1.0)*. It casts shadows and provides realistic illumination with a diffuse color of *(0.8, 0.8, 0.8)* and a specular color of *(0.2, 0.2, 0.2)*.
- Ground: A static *10x10* ground plane acts as the physical foundation for the simulation to ensure the tracker has a surface to stand on. It has a grey ambient and diffuse material color of *(0.8, 0.8, 0.8)*.

2.2. The Mechanical Links

The physical structure of the solar tracker model (named *solar_tracker*) is built using discrete rigid bodies known as links. Each link possesses specific geometric, visual, and inertial properties that dictate how it looks and behaves physically within the simulation.

- Base Pole: A vertical cylinder with a radius of *0.1m* and a length of *1.0m*, which is centered at pose *(0, 0, 0.5)* just above the top of the base pole. It is rendered with a dark grey material (*ambient 0.2, 0.2, 0.2*) and serves as the stationary support structure for the system.

- Solar Panel (*panel_link*): A rectangular box sized $2.0 \times 1.0 \times 0.05m$, positioned at pose $(0, 0, 1.05)$ just above the top of the base pole. It is rendered in blue (*ambient 0.0, 0.2, 0.8*) to represent the solar array. It has defined inertial properties that include a mass of $10.0Kg$ and a diagonal inertia tensor with $ixx=1.0$, $iyy=2.0$, and $izz=3.0kg.m^2$, which simulate realistic weight and rotational momentum as the panel moves.

2.3. Joints and Kinematics

To create a functional mechanism, the individual links must be connected using joints that define how they can move relative to one another. These joints constrain the motion of the links to mimic real-world mechanical assemblies.

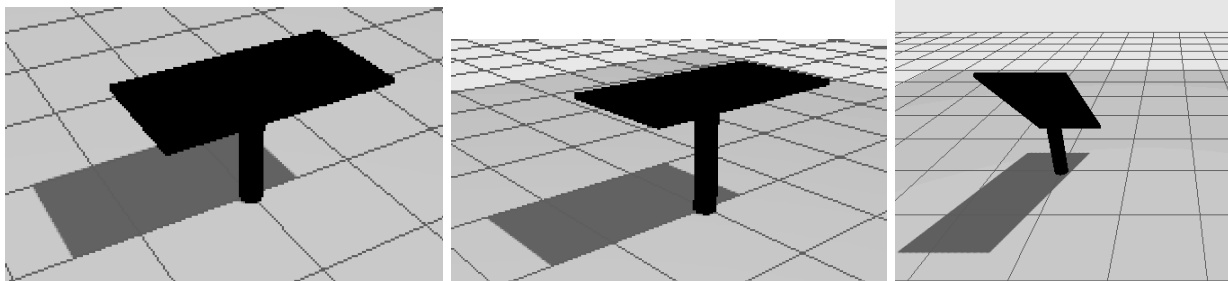
- Fixed Joint (*fixed_to_ground*): A joint that permanently anchors the base pole to the simulation world (parent: *world*, child: *base_pole*) so the structure remains stationary.
- Revolute Joint (*tracking_joint*): A joint that connects the base pole (parent) to the solar panel (child). This joint restricts the panel to rotate around a single axis (the Y-axis, defined by $xyz= 0 \ 1 \ 0$) and includes mechanical limits restricting the rotation between -1.57 and 1.57 radians (approximately $\pm 90^\circ$). This joint also enforces a maximum effort of $100.0 \ N.m$ and a maximum velocity of $1.0 \ rad/sec$. It incorporates physical dynamics such as a friction of $0.1 \ N.m$ and a damping of $0.5 \ N.m.s/rad$ to make the mechanical movement realistic.

2.4. Actuation and Sensing (Plugins)

In order to make the mechanical system controllable, Gazebo uses system plugins. These plugins act as the sensors and motors of the virtual robot, which allows external scripts to read physical data and send movement commands.

- Joint Position Controller (*gz-sim-joint-position-controller-system*): This plugin functions as the system's low-level actuator and PID controller. It targets the *tracking_joint* and listens for position commands on topic */model/solar_tracker/joint/tracking_joint/cmd_pos*. It applies the necessary torque effort to the revolute joint using the optimal PID gains that you need to calculate.
- Joint State Publisher (*gz-sim-joint-state-publisher-system*): This plugin acts as the system's sensor by continuously broadcasting the actual, real-time physical position of the tracking joint back to the simulation environment to make the joint data available for monitoring and control.

A couple of images of this solar tracking system in Gazebo are illustrated below.



3. Control System Design

In this phase of the project, you will design and implement a control system for the solar tracker. You should write a Python script that acts as a high-level controller for the system.

1. Your script should generate a continuous mathematical trajectory that represents the sun's path across the sky over a 25-second simulation period. You should implement the following reference function in your code as the sun's path:

$$\theta_{ref}(t) = -\cos\left(t \cdot \frac{\pi}{T}\right)$$

where T is the total duration (**25 seconds**). Your script will use this function to continuously calculate the target angle that the mechanical panel must strive to match at any given moment.

2. You should identify the optimal controller gains for the controller. In this respect, you should first use the physical parameters defined in your SDF file (e.g., the panel's moment of inertia, the joint's viscous damping coefficient), and derive the differential equation governing the rotational motion of the solar panel. You should model the system as a second-order rotational mass-damper system, such as follows:

$$I\ddot{\theta} + b\dot{\theta} = \tau$$

where I is the moment of inertia, b is the viscous friction, and τ is the applied torque.

3. Convert the system into a state-space model and obtain the controller gains for a state-feedback controller so that the system's response is critically damped. *Note that obtaining the state-feedback gains is identical to implementing a PD controller.* Therefore, your obtained gains are indeed the necessary proportional and derivative gains of a PD controller, which you can seamlessly implement using the Gazebo built-in functionalities.

4. Scenarios

There are five scenarios in total. The first three scenarios are mandatory, however, the last two scenarios are optional and have bonus points.

1. **Scenario #1:** This is the original scenario as explained above. You design a solar tracking panel that tracks the sun's path. Make sure you include a figure that shows the tracking performance of your controller, along with reporting the MAE (Mean Absolute Error) and RMSE (Root Mean Squared Error) of the tracking error.
2. **Scenario #2:** In this scenario, you add the wind disturbance to the system and evaluate the tracking performance when the solar panel is exposed to wind disturbance. Your script should dynamically calculate the wind torque by combining three elements: a continuous base oscillation (like a sine wave) to simulate rhythmic wind, high-frequency random noise for turbulence, and occasional high-magnitude spikes for sudden gusts. You should continuously apply this calculated disturbance torque directly to the *tracking joint*.

3. **Scenario #3:** In this scenario, you are required to consider the sensor inaccuracy. In order to do this, instead of relying on perfect position data, your script should dynamically corrupt the joint angle readings by combining three imperfections: Gaussian noise, quantization (rounding the continuous angle to discrete steps to mimic a low-resolution digital encoder) and slow drift (a time-varying, sinusoidal offset).
4. **Scenario #4 (optional, 2.5% bonus points):** In this scenario, you simulate the saturation in actuators. In order to do so, you should apply limits to the maximum torque the motor can deliver (e.g., 8.0 N.m). If your control system demands more torque than this limit under heavy loads, the actuator will saturate and the delivered torque will be clamped at its absolute maximum value. You should also restrict the maximum rotational speed of the joint to a slow rate (e.g., 0.2 rad/second). This simulates a slow mechanical gearbox that prevents the panel from slewing quickly to catch up with the sun's trajectory.
5. **Scenario #5 (optional, 2.5% bonus points):** In this scenario, you should model the effect of a mechanical gearbox on your tracking system. In the real world, solar trackers use small, low-torque motors paired with gearboxes to generate enough rotational force. You should simulate a weak motor connected to a gearbox with a 10:1 gear ratio, and calculate the actual angular acceleration based on this amplified motor torque.

5. Deliverables

You should deliver the following items. Your project groups are the same as your lab groups. Each group will submit only a single .zip file containing the following two items.

1. **Scenarios.** In this case, for each implemented scenario, you should create a folder, e.g., Scenario #1. In each folder, you should include the .sdf file as well as your Python script.
2. **Project Report.** You should write a 15-page report for this project. If you need extra pages, you should add the supplementary materials to the appendix section of your report. Your report should include the following sections:
 - a. Abstract: briefly describe the project, its objectives, and your observations.
 - b. Introduction: describe the problem that your project is trying to solve, its background, what are the current technologies, and how you are going to address the problem through this project.
 - c. Solar Tracking System: you should discuss the implementation process of your design, including the mechanical system design, the controller design, mathematical modeling of the system, the process of obtaining the controller gains, etc.
 - d. Results: you should present your results for each of the scenarios with graphs, tables, and present the tracking error in each scenario, and discuss your observations and justifications.
 - e. Conclusion: conclude your report.